T106 COMPUTER PROGRAMMING

Unit – II

Problem solving techniques – Program – Program development cycle – Algorithm design – Flowchart - Pseudo code. Introduction to C – History of C – Importance of C - C tokens – data types – Operators and expressions – I/O functions.

2-Marks

1)Explain the steps in the Program Development Cycle. (dec-18)

Program development Life Cycle(PDLC): The process of creating application **programs**. The process containing the five **phases** of **program development**:

- Analyzing
- Designing
- Coding
- Debugging and Testing
- Implementing and Maintaining application software.

2) Bring out the Importance of C Programming. (dec-18

- C is highly portable language. i.e. code written in one machine can be moved other which is very important and powerful feature.
- C is robust language and has rich set of build-in-functions, data types and operators which can be used to write any complex program.
- C language has its ability to extend itself. A c program is basically a collection of functions that are supported by the C library.

3) State the various primary data types available in C. (may-18)

Primitive types are also known as pre-defined or basic **data types.**These are fundamental **data types in C** namely integer(int), floating point(float), character (char) and void .

4)Define an Identifier. (may-18)

- An **identifier** is nothing but a name assigned to an element in a program. An **identifiers** are used to identify a particular element in a program.
- An identifier can be composed of letters such as uppercase, lowercase letters, underscore, digits, but the starting letter should be either an alphabet or an underscore.
- "**Identifiers**" or "symbols" are the names for variables, types, functions, and labels in your program.
- **Identifier** names must differ in spelling and case from any keywords. You cannot use keywords as **identifiers**.

5)Define Pseudocode. (dec-17, may-14)

- The **pseudo code** consists of words and phrases that make **pseudo code** looks similar to the program but not a program.
- Pseudo codes are written with respect to a programming language but grammar is not strictly followed.
- Pseudocode is made up of two words, 'pseudo' and 'code'. Pseudo means false. It is the false code of a program.

6)What do you mean by C-Tokens? Give an example. (dec-17)

- **C Tokens** are the smallest building block or smallest unit of a **C** program.
- The compiler breaks a **program** into the smallest possible units (**tokens**) and proceeds to the various stages of the compilation.
- Eg.: Keywords, Identifiers, Constant, Strings, Operators, etc.

7)Define Flow chart. (may-17)

- A **flowchart** is simple graphical representation of a program.
- It shows steps in sequential order and is widely used in presenting the **flow** of algorithms, workflow or processes.
- Typically, a **flowchart** shows the steps as boxes of various kinds, and their order by connecting them with arrows.

8)What are the rules for naming a variables? (may-17)

- All **variable names** must begin with a letter of the alphabet or underscore( ).
- After the first initial letter, **variable names** can also contain letters and numbers.
- Uppercase characters are distinct from lowercase characters.
- You cannot use a C++ keyword as a **variable** name.
- A **variable name** can only have letters (both uppercase and lowercase letters), digits and underscore.
- The first letter of a **variable** should be either a letter or an underscore.
- There is no **rule** on how long a **variable name** (identifier) can be.

9)What is Operator Precedence? (nov-16, may-14)

- **Operator precedence** determines the grouping of terms in an expression and decides how an expression is evaluated.
- Certain **operators** have higher **precedence** than others.
- For example, the multiplication **operator** has a higher **precedence** than the addition **operator**.

10)What are the benefits of Pseudocode? (may-16)

- It is easy to understand, even for non-programmers. It allows the designer to focus on main logic without being distracted by programming languages syntax.

- Since, it is language independent, it can be translated to any computer language code.
- It allows designer to express logic in plain natural language

11)What are Escape Sequences in c? (may-16)

- In the C language, an escape sequence is a series of 2 or more characters, starting with a **backslash** (\).
- Some Escape sequence are:
  \n(New Line) \t(Horizontal Tab) \r(carriage return) \v(Vertical tab)

12)What are the Characteristics of an algorithm? (jan16)

(i)Input (ii) Output (iii) Finiteness (iv) Definiteness (v) Effectiveness (vi) Independent

13)Distinguish between Logical and Bitwise operators. (jan-16)

Logical Operator	Bitwise Operator
Logical operators work on boolean expressions and return boolean values	**Bitwise operators** work on binary digits of integer values and returns an integer.
It has two values true and false.	They are used in bit level **programming**. These **operators** are used to manipulate bits of an integer expression.
|| (OR), && (AND), ! (NOT)	**Binary** AND,OR,XOR

11 Marks

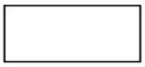
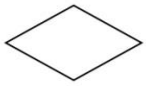
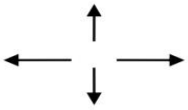
1.Write an example, explain the following: a) Flowchart b) Pseudocode (dec-18)

a) Flowchart:-

Flowchart uses different symbols to design a solution to a problem. it is another commonly used programming tool. By looking at a flowchart one can understand the operations and sequence of operations performed in a system. Flowchart is often considered as a blueprint of a design used for solving a specific problem.

Flowchart

It is diagrammatic / Graphical representation of sequence of steps to solve a problem. To draw a flowchart following standard symbols are use.

Symbol Name	Symbol	function
Oval		Used to represent start and end of flowchart
Parallelogram		Used for input and output operation
Rectangle		Processing: Used for arithmetic operations and data-manipulations
Diamond		Decision making. Used to represent the operation in which there are two/three alternatives, true and false etc
Arrows		Flow line Used to indicate the flow of logic by connecting symbols
Circle		Page Connector

b) Pseudocode

Definition: Pseudocode is an informal way of programming description that does not require any strict programming language syntax or underlying technology considerations.

It is used for creating an outline or a rough draft of a program.

Pseudocode summarizes a program's flow, but excludes underlying details.

System designers write pseudocode to ensure that programmers understand a software project's requirements and align code accordingly.

Algorithm: It's an organized logical sequence of the actions or the approach towards a particular problem. A programmer implements an algorithm to solve a problem. Algorithms are expressed using natural verbal but somewhat technical annotations.

Description: Pseudocode is not an actual programming language. So it cannot be compiled into an executable program. It uses short terms or simple English language syntaxes to write code for programs before it is actually converted into a specific programming language. This is done to identify top level flow errors, and understand the programming data flows that the final program is going to use. This definitely helps save time during actual programming as conceptual errors have been already corrected.

Advantages of pseudocode

• Pseudocode is understood by the programmers of all types.

• it enables the programmer to concentrate only on the algorithm part of the code development.
• It cannot be compiled into an executable program.
- Improves the readability of any approach. It's one of the best approaches to start implementation of an algorithm.
- Acts as a bridge between the program and the algorithm or flowchart. Also works as a rough documentation, so the program of one developer can be understood easily when a pseudo code is written out. In industries, the approach of documentation is essential. And that's where a pseudo-code proves vital.
- The main goal of a pseudo code is to explain what exactly each line of a program should do, hence making the code construction phase easier for the programmer.

Disadvantages of Pseudocode

- Pseudocode does not provide a visual representation of the logic of programming.
- There are no proper format for writing the for pseudocode.
- In Pseudocode their is extra need of maintain documentation.
- In Pseudocode their is no proper standard very company follow their own standard for writing the pseudocode.

How to write a Pseudo-code?

1. Arrange the sequence of tasks and write the pseudocode accordingly.
2. Start with the statement of a pseudo code which establishes the main goal or the aim.

Example:

This program will allow the user to check the number whether it's even or odd.

- The way the if-else, for, while loops are indented in a program, indent the statements likewise, as it helps to comprehend the decision control and execution mechanism. They also improve the readability to a great extent.

Example:

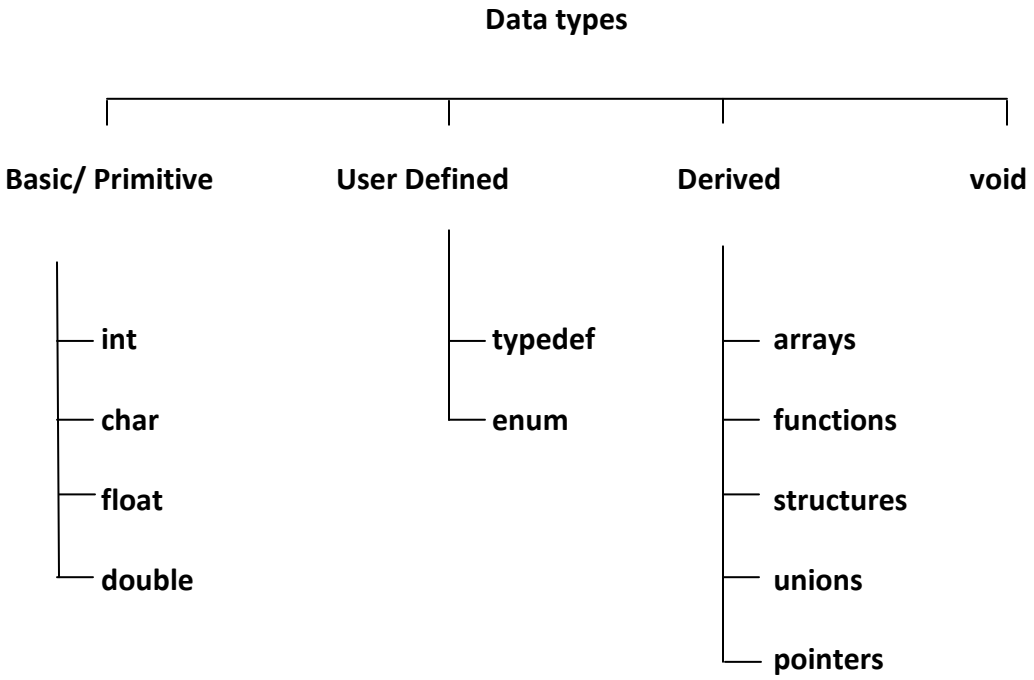
```
if "1"
   print response
      "I am case 1"

if "2"
   print response
      "I am case 2"
```

1. Use appropriate naming conventions. The human tendency follows the approach to follow what we see. If a programmer goes through a pseudo code, his approach will be the same as per it, so the naming must be simple and distinct.
2. Use appropriate sentence casings, such as CamelCase for methods, upper case for constants and lower case for variables.
3. Elaborate everything which is going to happen in the actual code. Don't make the pseudo code abstract.
4. Use standard programming structures such as 'if-then', 'for', 'while', 'cases' the way we use it in programming.
5. Check whether all the sections of a pseudo code is complete, finite and clear to understand and comprehend.
6. Don't write the pseudo code in a complete programmatic manner. It is necessary to be simple to understand even for a layman or client, hence don't incorporate too many technical terms.

2. Explain the various basic data types available in C programming. (dec-18, dec-17, may-14)

DATA TYPES

Data Type is used to define the type of value to be used in a Program. Based on the type of value specified in the program specified amount of required Bytes will be allocated to the variables used in the program. Data types are broadly classified into four main types. They are as follows.



Primary Data Type

Integers are represented as int, character are represented as char, floating point value are represented as float, double precision floating point are represented as double and finally void are primary data types. Primary data type offers extended data types. longint, long double are extended data types.

Integer Data Type

Integer data type can store only the whole numbers.

Name	C Representation	Size (bytes)	Range	Format specifier
Integer	int	2	-32768 to 32767	%d(%i)
Short Integer	short int / short	2	-32768 to 32767	%d(%i)
Long Integer	long int / long	4	-2147483648 to 2147483647	%ld
Signed Integer	signed int	2	-32768 to 32767	%d(%i)
Unsigned Integer	unsigned int	2	0 to 65535	%u
Signed Short Integer	unsigned short int / short	2	-32768 to 32767	%d
Unsigned Short Integer	unsigned short int / short	2	0 to 65535	%u
Signed Long Integer	signed long int / long	4	-2147483648 to 2147483647	%ld
Unsigned Long Integer	unsigned long int / long	4	0 to 4294967295	%lu

Floating Point Data Type

Floating Point data types are also known as Real Numbers. It can store only the real numbers (decimal numbers) with 6 digits precision.

Name	C Representation	Size (bytes)	Range	Format Delimiter
Float	Float	4	3.4 e-38 to 3.4 e+38	%f(%e)
Double	Double	8	1.7e-308 to 1.7e+308	%lf
Long Double	long double	10	3.4 e-4932 to 1.1 e+4932	%lf

Character Data Type

Normally a single character is defined as char data type. It is specified by the keyword char. Char data type uses 8 bits for storage. Char may be signed char or unsigned char.

Name	C Representation	Size (bytes)	Range	Format Delimiter
Character	Char	1	-128 to 127	%c
Signed Character	signed char	1	-128 to 127	%c
Unsigned Character	unsigned char	1	0 to 255	%c

Empty Data Type

void is the empty data type. This data type is used before main function in a C Language. The word void refers no return data type. It is used before the main function to specify the type of function. If a function is of type void it does not return any value to the calling function.

3.With suitable examples, describe the following operators. a) Conditional operator b) Relational operator c) Logical operators. d)Arithmetic operators. e) Bitwise operators. (may-18, may-17, nov-16, may-16, jan-15)

Operators:

Operators are symbols that trigger an action when applied to C variables and other objects.
The data items on which operators act upon are called operands.
Depending on the number of operands that an operator can act upon, operators can be classified as follows:

Unary Operators: Those operators that require only a single operand to act upon are known as unary operators. For Example increment and decrement operators

Binary Operators: Those operators that require two operands to act upon are called binary operators.

Binary operators are classified into :

1. Arithmetic operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Conditional Operators
6. Bitwise Operators

Arithmetic Operators

An arithmetic operator performs mathematical operations such as addition, subtraction, multiplication, division etc on numerical values (constants and variables).

Operator	Meaning of Operator
+	addition or unary plus
-	subtraction or unary minus
*	multiplication

Operator	Meaning of Operator
/	division
%	remainder after division (modulo division)

Example: Arithmetic Operators

```
// Working of arithmetic operators
#include <stdio.h>
int main()
{
   int a = 9,b = 4, c;

   c = a+b;
   printf("a+b = %d \n",c);
   c = a-b;
   printf("a-b = %d \n",c);
   c = a*b;
   printf("a*b = %d \n",c);
   c = a/b;
   printf("a/b = %d \n",c);
   c = a%b;
   printf("Remainder when a divided by b = %d \n",c);

   return 0;
}
```

Output

```
a+b = 13
a-b = 5
a*b = 36
a/b = 2
Remainder when a divided by b=1
```

The operators +, - and * computes addition, subtraction, and multiplication respectively as you might have expected.

In normal calculation, 9/4 = 2.25. However, the output is 2 in the program.
It is because both the variables a and b are integers. Hence, the output is also an integer. The compiler neglects the term after the decimal point and shows answer 2 instead of 2.25.
The modulo operator % computes the remainder. When a=9 is divided by b=4, the remainder is 1. The % operator can only be used with integers.
Suppose a = 5.0, b = 2.0, c = 5 and d = 2. Then in C programming,

```
// Either one of the operands is a floating-point number
a/b = 2.5
a/d = 2.5
c/b = 2.5

// Both operands are integers
c/d = 2
```

Relational Operators

A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0.
Relational operators are used in decision making and loops.

Operator	Meaning of Operator	Example
==	Equal to	5 == 3 is evaluated to 0
>	Greater than	5 > 3 is evaluated to 1
<	Less than	5 < 3 is evaluated to 0
!=	Not equal to	5 != 3 is evaluated to 1
>=	Greater than or equal to	5 >= 3 is evaluated to 1
<=	Less than or equal to	5 <= 3 is evaluated to 0

Example: Relational Operators

```
// Working of relational operators
#include <stdio.h>
```

```
int main()
{
   int a = 5, b = 5, c = 10;

   printf("%d == %d is %d \n", a, b, a == b);
   printf("%d == %d is %d \n", a, c, a == c);
   printf("%d > %d is %d \n", a, b, a > b);
   printf("%d > %d is %d \n", a, c, a > c);
   printf("%d < %d is %d \n", a, b, a < b);
   printf("%d < %d is %d \n", a, c, a < c);
   printf("%d != %d is %d \n", a, b, a != b);
   printf("%d != %d is %d \n", a, c, a != c);
   printf("%d >= %d is %d \n", a, b, a >= b);
   printf("%d >= %d is %d \n", a, c, a >= c);
   printf("%d <= %d is %d \n", a, b, a <= b);
   printf("%d <= %d is %d \n", a, c, a <= c);

   return 0;
}
```

Output

```
5 == 5 is 1
5 == 10 is 0
5 > 5 is 0
5 > 10 is 0
5 < 5 is 0
5 < 10 is 1
5 != 5 is 0
5 != 10 is 1
5 >= 5 is 1
```

```
5 >= 10 is 0
5 <= 5 is 1
5 <= 10 is 1
```

Logical Operators

An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false. Logical operators are commonly used in decision making in C programming.

Operator	Meaning	Example
&&	Logical AND. True only if all operands are true	If c = 5 and d = 2 then, expression ((c==5) && (d>5)) equals to 0.
||	Logical OR. True only if either one operand is true	If c = 5 and d = 2 then, expression ((c==5) || (d>5)) equals to 1.
!	Logical NOT. True only if the operand is 0	If c = 5 then, expression !(c==5) equals to 0.

Example: Logical Operators

```
// Working of logical operators

#include <stdio.h>
int main()
{
   int a = 5, b = 5, c = 10, result;

   result = (a == b) && (c > b);
   printf("(a == b) && (c > b) is %d \n", result);

   result = (a == b) && (c < b);
   printf("(a == b) && (c < b) is %d \n", result);

   result = (a == b) || (c < b);
```

```
    printf("(a == b) || (c < b) is %d \n", result);

    result = (a != b) || (c < b);
    printf("(a != b) || (c < b) is %d \n", result);
    result = !(a != b);
    printf("!(a != b) is %d \n", result);

    result = !(a == b);
    printf("!(a == b) is %d \n", result);
    return 0;
}
```

Output

```
(a == b) && (c > b) is 1
(a == b) && (c < b) is 0
(a == b) || (c < b) is 1
(a != b) || (c < b) is 0
!(a != b) is 1
!(a == b) is 0
```

Explanation of logical operator program

1. `(a == b) && (c > 5)` evaluates to 1 because both operands `(a == b)` and `(c > b)` is 1 (true).
2. `(a == b) && (c < b)` evaluates to 0 because operand `(c < b)` is 0 (false).
3. `(a == b) || (c < b)` evaluates to 1 because `(a = b)` is 1 (true).
4. `(a != b) || (c < b)` evaluates to 0 because both operand `(a != b)` and `(c < b)` are 0 (false).
5. `!(a != b)` evaluates to 1 because operand `(a != b)` is 0 (false). Hence, !(a != b) is 1 (true).
6. `!(a == b)` evaluates to 0 because `(a == b)` is 1 (true). Hence, `!(a == b)` is 0 (false).

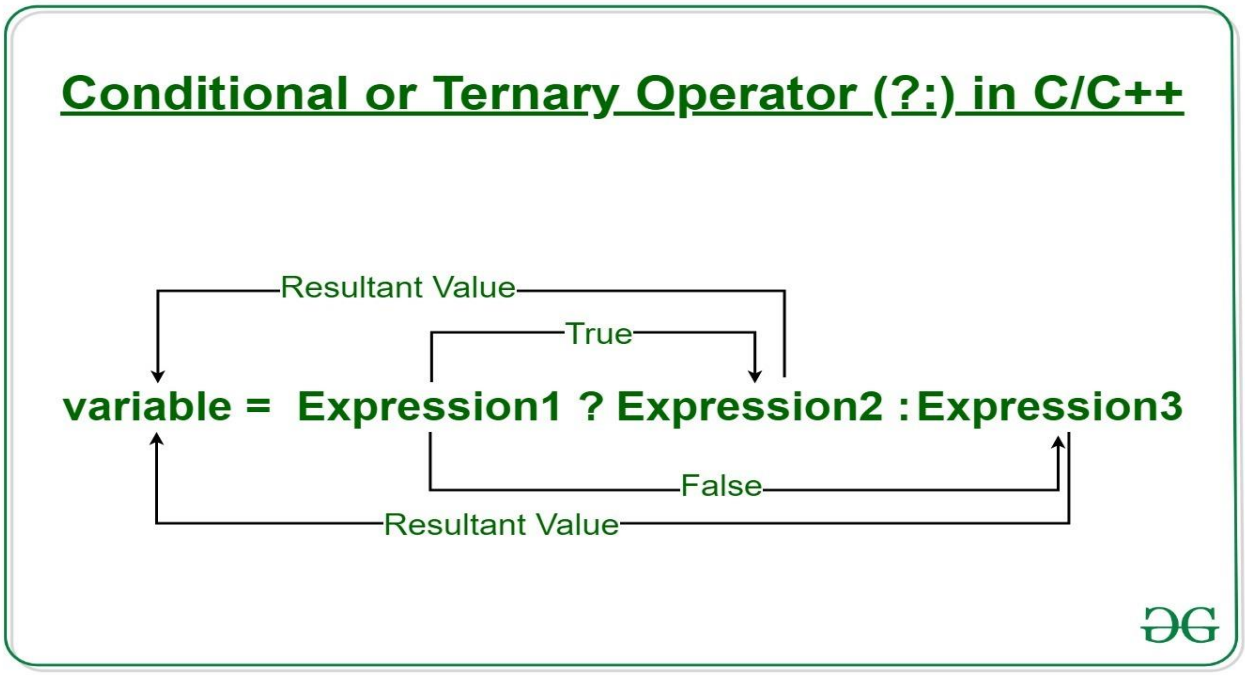
Bitwise Operators

During computation, mathematical operations like: addition, subtraction, multiplication, division, etc are converted to bit-level which makes processing faster and saves power.
Bitwise operators are used in C programming to perform bit-level operations.

Operators	*Meaning of operators*
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
~	Bitwise complement
<<	Shift left
>>	Shift right

Conditional or Ternary Operator (?:) in C

The conditional operator is kind of similar to the if-else statement as it does follow the same algorithm as of if-else statement but the conditional operator takes less space and helps to write the if-else statements in the shortest way possible.



Syntax:

The conditional operator is of the form

variable = Expression1 ? Expression2 : Expression3

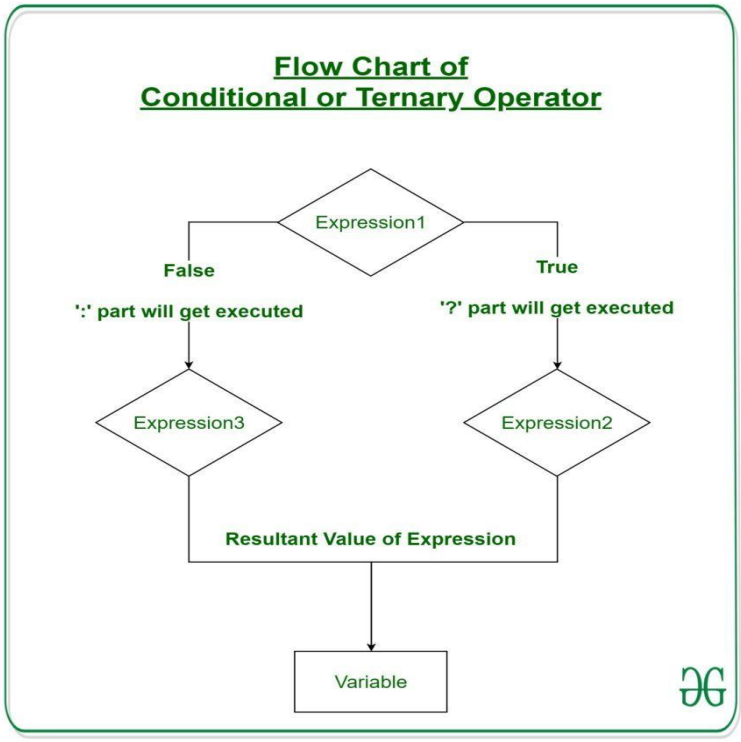
It can be visualized into if-else statement as:

```
if(Expression1)
{
   variable = Expression2;
}
else
{
   variable = Expression3;
}
```

Since the Conditional Operator '?:' takes three operands to work, hence they are also called **ternary operators**.

Working:

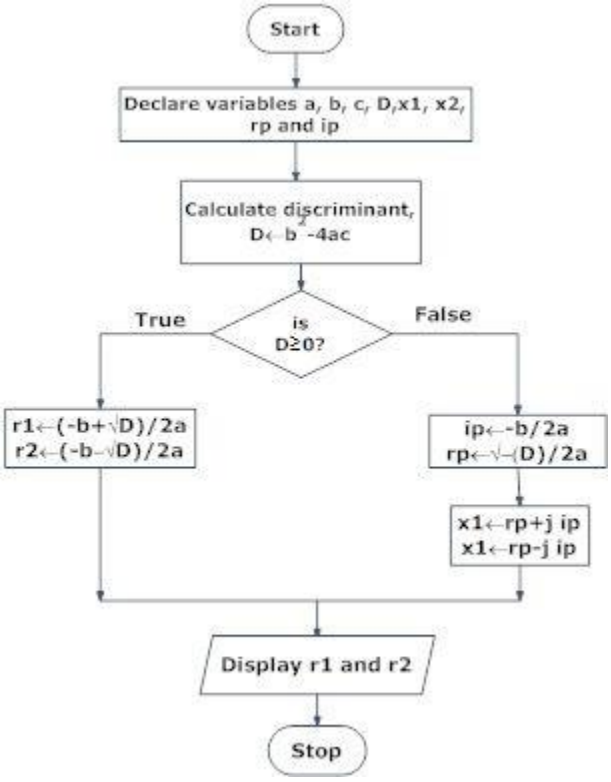
Here, **Expression1** is the condition to be evaluated. If the condition(**Expression1**) is True then **Expression2** will be executed and the result will be returned. Otherwise, if the condition(**Expression1**) is false then **Expression3** will be executed and the result will be returned.



Example: Program to Store the greatest of the two Number.

```
// C program to find largest among two
// numbers using ternary operator
 #include <stdio.h>
 int main()
{
   int m = 5, n = 4;
    (m > n) ? printf("m is greater than n that is %d > %d", m, n)
          : printf("n is greater than m that is %d > %d", n, m);
    return 0;    }
```

4. Draw the flow chart for finding the roots of a quadratic equation. Write the program (MAY 2016)



Program:

```
#include<stdio.h>
#include<math.h>  // it is used for math calculation
#include<conio.h>
void main()
{
   float x, y, z, det, root1, root2, real, img;
   printf("\n Enter the value of coefficient x, y and z: \n ");
   scanf("%f %f %f", &x, &y, &z);
   // define the quadratic formula of the nature of the root
   det = y * y - 4 * x * z;
   // defines the conditions for real and different roots of the quadratic equation
   if (det > 0)
   {
   root1 = (-y + sqrt(det)) / (2 * x);
   root2 = (-y + sqrt(det)) / (2 * x);
```

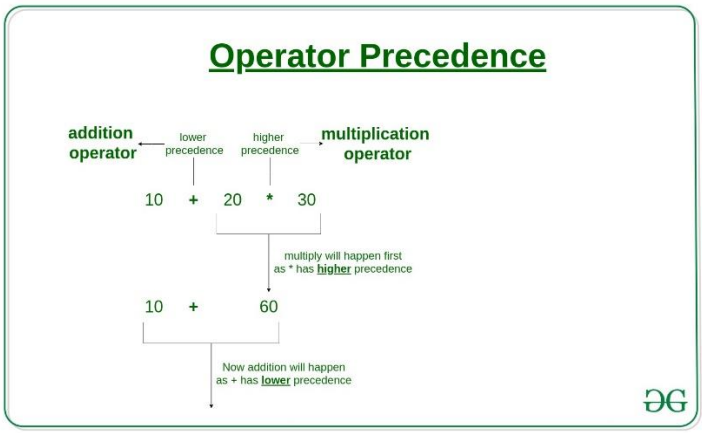
```
    printf("\n Value of root1 = %.2f and value of root2 = %.2f", root1, root2);
    }
    // elseif condition defines both roots ( real and equal root) are equal in the quadratic equ
ation
    else if (det == 0)
    {
        root1 = root2 = -y / (2 * x); // both roots are equal;
        printf("\n Value of root1 = %.2f and Value of root2 = %.2f", root1, root2);
    }
    // if det < 0, means both roots are real and imaginary in the quadratic equation.
    else {
        real = -y / (2 * x);
        img = sqrt(-det) / (2 * x);
        printf("\n value of root1 = %.2f + %.2fi and value of root2 = %.2f -
 %.2fi ", real, img, real, img);
    }
    getch();
    }
```

5. Define Precedence and associativity. Give the precedence and associativity of the operators. (Jan 2016)

Operator precedence determines which operator is performed first in an expression with more than one operators with different precedence.

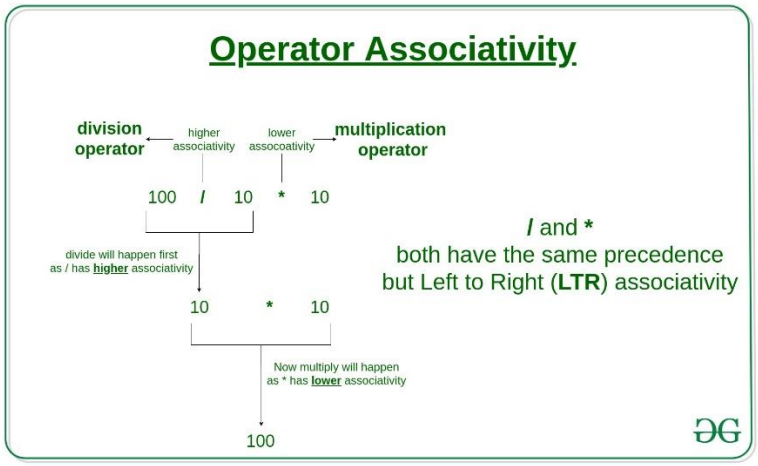
For example: Solve

10 + 20 * 30

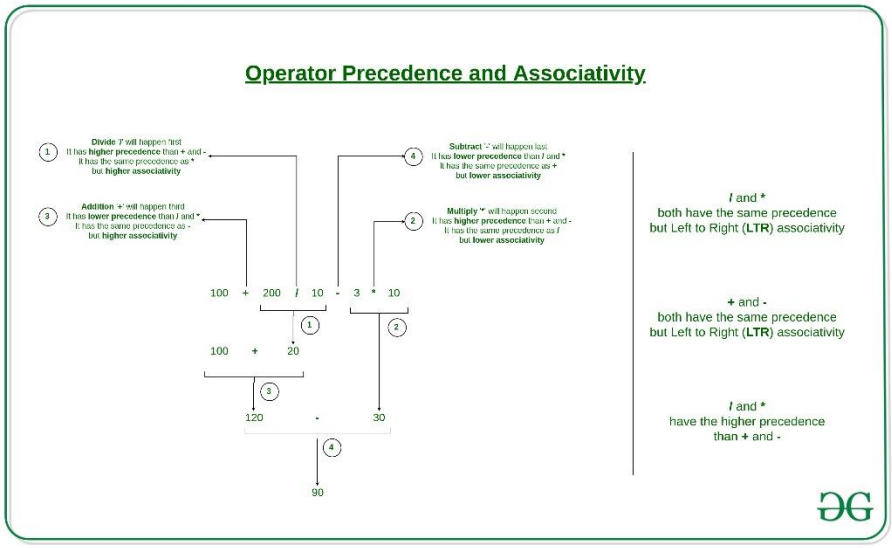


10 + 20 * 30 is calculated as **10 + (20 * 30)**
and not as **(10 + 20) * 30**
Operators Associativity is used when two operators of same precedence appear in an expression. Associativity can be either **L**eft **t**o **R**ight or **R**ight **t**o **L**eft.

For example: '*' and '/' have same precedence and their associativity is **L**eft **t**o **R**ight, so the expression "100 / 10 * 10" is treated as "(100 / 10) * 10".



For example: Solve
100 + 200 / 10 - 3 * 10



1) Associativity is only used when there are two or more operators of same precedence. The point to note is associativity doesn't define the order in which operands of a single operator are evaluated. For example, consider the following program, associativity of the + operator is left to right, but it doesn't mean f1() is always called before f2(). The output of the following program is in-fact compiler dependent.

// Associativity is not used in the below program.

```
// Output is compiler dependent.

#include <stdio.h>

int x = 0;
int f1()
{
   x = 5;
   return x;
}
int f2()
{
   x = 10;
   return x;
}
int main()
{
   int p = f1() + f2();
   printf("%d ", x);
   return 0;
}
```

2) All operators with the same precedence have same associativity
This is necessary, otherwise, there won't be any way for the compiler to decide evaluation order of expressions which have two operators of same precedence and different associativity. For example + and – have the same associativity.

3) Precedence and associativity of postfix ++ and prefix ++ are different
Precedence of postfix ++ is more than prefix ++, their associativity is also different. Associativity of postfix ++ is left to right and associativity of prefix ++ is right to left.

4) Comma has the least precedence among all operators and should be used carefully For example consider the following program, the output is 1..

```
#include <stdio.h>
int main()
{
   int a;
   a = 1, 2, 3; // Evaluated as (a = 1), 2, 3
   printf("%d", a);
   return 0;
}
```

5) There is no chaining of comparison operators in C

For example consider the following program. The output of following program is "FALSE".

```
#include <stdio.h>
int main()
{
   int a = 10, b = 20, c = 30;

   // (c > b > a) is treated as ((c  > b) > a), associativity of '>'
   // is left to right. Therefore the value becomes ((30 > 20) > 10)
   // which becomes (1 > 20)
   if (c > b > a)
      printf("TRUE");
   else
      printf("FALSE");
   return 0;
}
```

Precedence and associativity table.

Operator	Description	Associativity
( ) [ ] . -> ++ —	Parentheses (function call) Brackets (array subscript) Member selection via object name Member selection via pointer Postfix increment/decrement	left-to-right
++ — + – ! ~ (*type*) * & sizeof	Prefix increment/decrement Unary plus/minus Logical negation/bitwise complement Cast (convert value to temporary value of *type*) Dereference Address (of operand) Determine size in bytes on this implementation	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ –	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right

< <= > >=	Relational less than/less than or equal to Relational greater than/greater than or equal to	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
|	Bitwise inclusive OR	left-to-right
&&	Logical AND	left-to-right
||	Logical OR	left-to-right
? :	Ternary conditional	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left
,	Comma (separate expressions)	left-to-right

It is good to know precedence and associativity rules, but the best thing is to use brackets, especially for less commonly used operators (operators other than +, -, *.. etc). Brackets increase the readability of the code as the reader doesn't have to see the table to find out the order.

Explain the method of evaluating of expression in C with examples (May 2014)

Expression Evaluation in C

- An expression in C is defined as 2 or more operands are connected by one operator and which can also be said to a formula to perform any operation.
- An operand is a function reference, an array element, a variable, or any constant. An operator is symbols like "+", "-", "/", "*" etc.

- Now expression evaluation is nothing but operator precedence and associativity. Expression precedence in C tells you which operator is performed first, next, and so on in an expression with more than one operator with different precedence.
- This plays a crucial role while we are performing day to day arithmetic operations.
- If we get 2 same precedences appear in an expression, then it is said to be "Associativity".
- Now in this case we can calculate this statement either from Left to right or right to left because this both are having the same precedence.

Types of Expression Evaluation in C

In C there are 4 types of expressions evaluations

1. Arithmetic expressions evaluation
2. Relational expressions evaluation
3. Logical expressions evaluation
4. Conditional expressions evaluation

Every expression evaluation of these 4 types takes certain types of operands and used a specific type of operators. The result of this expression evaluation operation produces a specific value.

Example to Implement Expression Evaluation in C

Below are some examples mentioned:

1. Arithmetic expression Evaluation

Addition (+), Subtraction(-), Multiplication(*), Division(/), Modulus(%), Increment(++) and Decrement(–) operators are said to "Arithmetic expressions". These operators work in between operands. like A+B, A-B, A–, A++ etc. While we perform the operation with these operators based on specified precedence order as like below image.

```
A+B*C/D-E%F
```

Arithmetic Expression evaluation order:

*, /, %	3	Left to right
+, -	4	Left to right

Code:

```
//used to include basic C libraries
#include <stdio.h>
//main method for run c application
int main()
{
//declaring variables
int a,b,result_art_eval;
//Asking the user to enter 2 numbers
printf("Enter 2 numbers for Arithmetic evaluation operation\n");
//Storing 2 numbers in varaibles a and b
```

```
scanf("%d\n%d",&a,&b);
//Arithmetic evaluation operations and its result displaying
result_art_eval = a+b*a/b-a%b;
printf("================ARITHMETIC EXPRESSION EVALUATION==============\n");
printf("Arithmetic expression evaluation of %d and %d is = %d \n",a,b,result_art_eval);
printf("============================================================");
return 0;
}
```

Output:

```
Enter 2 numbers for Arithmetic evaluation operation
10
2
================ARITHMETIC EXPRESSION EVALUATION==============
Arithmetic expression evaluation of 10 and 2 is = 20
============================================================
```

Explanation: As you can see in the above example arithmetic expression values evaluated based on precedence as the first *, followed by /, %, + and -.

2. Relational expression evaluation

== (equal to), != (not equal to), > (greater than), < (less than), >= (greater than or equal to), <= (less than or equal to) operators are said to “Relational expressions”. This operator works in between operands. Used for comparing purpose. Like A==B, A!=B, A>B, A<B etc. While we perform the operation with these operators based on specified precedence order as like the below image.

```
A==B || A!=B||A<B||A>B;
```

Relation expression evaluation order:

<, <=, >, >=, ==, !=	5	Left to right

Code:

```
//used to include basic C libraries
#include <stdio.h>
//main method for run c application
int main()
{
//declaring variables
int a,b;
int result_rel_eval;
//Asking the user to enter 2 numbers
printf("Enter 2 numbers for Relational evaluation operation\n");
//Storing 2 numbers in varaibles a and b
```

```
scanf("%d\n%d",&a,&b);
//Relational evaluation operations and its result displaying
//If we get result as 1 means the value is true and if it is 0 then value is false in C
result_rel_eval = (a<b||a<=b||a>=b||a>b||a!=b);
printf("================RELATIONAL EXPRESSION EVALUATION=============\n");
if(result_rel_eval==1)
{
printf("Relational expression evaluation of %d and %d is = %d \n",a,b,result_rel_eval);
}
else
{
printf("Relational expression evaluation of %d and %d is = %d \n",a,b,result_rel_eval);
}
printf("============================================================");
return 0;
}
```

Output:

```
Enter 2 numbers for Relational evaluation operation
10
20
================RELATIONAL EXPRESSION EVALUATION=============
Relational expression evaluation of 10 and 20 is = 1
============================================================
```

Explanation: As you can see in the above example relational expression values evaluated based on precedence as First <, followed by <=, >, >=, ==, !=.

3. Logical expression evaluation

&&(Logical and), ||(Logical or) and !(Logical not) operators are said to “Logical expressions”. Used to perform a logical operation. These operators work in between operands. Like A&&B, A||B, A!B etc. While we perform the operation with these operators based on specified precedence order as like the below image.

```
A&&B||B!A;
```

Logical expression evaluation order:

&&	6	Left to right
||	7	Left to right

Code:

```
//used to include basic C libraries
#include <stdio.h>
```

```
//main method for run c application
int main()
{
//declaring variables
int a,b;
int result_log_eval;
//Asking the user to enter 2 numbers
printf("Enter 2 numbers for Logical evaluation operation\n");
//Storing 2 numbers in varaibles a and b
scanf("%d\n%d",&a,&b);
//Logical evaluation operations and its result displaying
//If we get result as 1 means the value is true and if it is 0 then value is false in C
result_log_eval = (a||b&&a);
printf("================LOGICAL EXPRESSION EVALUATION==============\n");
if(result_log_eval==1)
{
printf("Logical expression evaluation of %d and %d is = %d \n",a,b,result_log_eval);
}
else
{
printf("Logical expression evaluation of %d and %d is = %d \n",a,b,result_log_eval);
}
printf(============================================================");
return 0;
}
```

Output:

```
Enter 2 numbers for Logical evaluation operation
10
25
================LOGICAL EXPRESSION EVALUATION==============
Logical expression evaluation of 10 and 25 is = 1
============================================================
```

Explanation: As you can see in the above example logical expression values evaluated based on precedence as the First &&, followed by || and !.

4. Conditional expression Evaluation

?(Question mark) and :(colon) are said to “Conditional expressions”. Used to perform a conditional check. It has 3 expressions first expression is condition. If it is true then execute

expression2 and if it is false then execute expression3. Like (A>B)?"A is Big":"B is Big". While we perform the operation with these operators based on specified precedence order as like the below image.

(X+2=10)?'true':'false';

Conditional expression Evaluation order:

?:	8	Right to left

Code:

```
//used to include basic C libraries
#include <stdio.h>
//main method for run c application
int main()
{
//declaring variables
int a,b,c;
int result_con_eval;
//Asking the user to enter 3 numbers
printf("Enter 3 numbers for Conditional evaluation operation\n");
//Storing 3 numbers in varaibles a, b and c
scanf("%d\n%d\n%d",&a,&b,&c);
//Conditional evaluation operations and its result displaying
result_con_eval=(a>b && a>b)? a: (b>a && b>c)? b:c;
printf("================CONDITIONAL EXPRESSION EVALUATION==============\n");
printf("Maximum value of %d, %d and %d is = %d \n",a,b,c,result_con_eval);
printf("============================================================");
return 0;
}
```

Output:

```
Enter 3 numbers for Conditional evaluation operation
10
20
5
================CONDITIONAL EXPRESSION EVALUATION==============
Maximum value of 10, 20 and 5 is = 20
============================================================
```

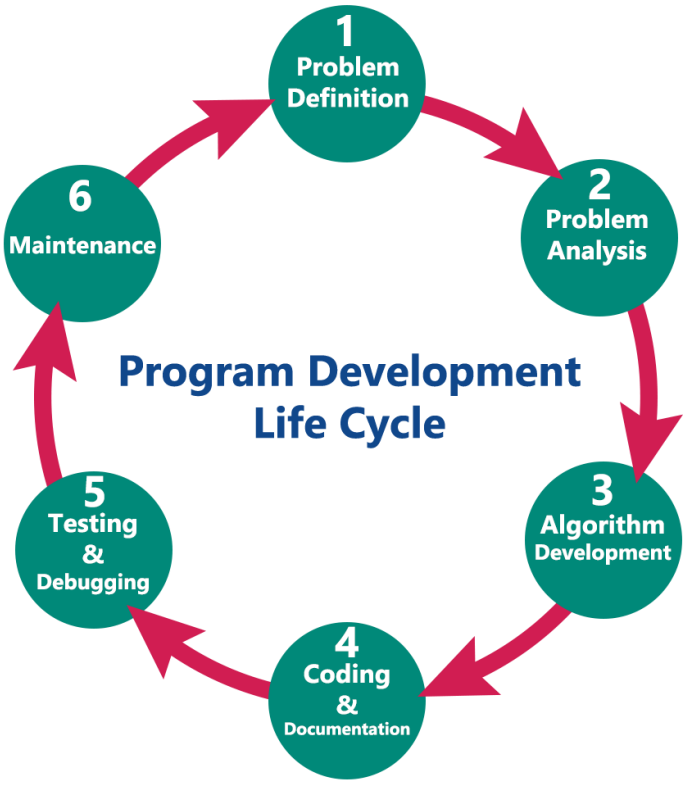
6.Explain the steps of software development with example. (may-16)

Discuss in detail about the steps involved in developing a program. Or explain the program development cycle (jan-16, jan-15, may-14)

Program Development Life Cycle

When we want to develop a program using any programming language, we follow a sequence of steps. These steps are called phases in program development. The program development life cycle is a set of steps or phases that are used to develop a program in any programming language. Generally, the program development life cycle contains 6 phases, they are as follows….

- Problem Definition
- Problem Analysis
- Algorithm Development
- Coding & Documentation
- Testing & Debugging
- Maintenance



1. Problem Definition

In this phase, we define the problem statement and we decide the boundaries of the problem. In this phase we need to understand the problem statement, what is our requirement, what should be the output of the problem solution. These are defined in this first phase of the program development life cycle.

2. Problem Analysis

In phase 2, we determine the requirements like variables, functions, etc. to solve the problem. That means we gather the required resources to solve the problem defined in the problem definition phase. We also determine the bounds of the solution.

3. Algorithm Development

During this phase, we develop a step by step procedure to solve the problem using the specification given in the previous phase. This phase is very important for program development. That means we write the solution in step by step statements.

4. Coding & Documentation

This phase uses a programming language to write or implement the actual programming instructions for the steps defined in the previous phase. In this phase, we construct the actual program. That means we write the program to solve the given problem using programming languages like C, C++, Java, etc.,

5. Testing & Debugging

During this phase, we check whether the code written in the previous step is solving the specified problem or not. That means we test the program whether it is solving the problem for various input data values or not. We also test whether it is providing the desired output or not.

6. Maintenance

During this phase, the program is actively used by the users. If any enhancements found in this phase, all the phases are to be repeated to make the enhancements. That means in this phase, the solution (program) is used by the end-user. If the user encounters any problem or wants any enhancement, then we need to repeat all the phases from the starting, so that the encountered problem is solved or enhancement is added.

Algorithm in C Language

Algorithm is a step-by-step procedure, which defines a set of instructions to be executed in a certain order to get the desired output. Algorithms are generally created independent of

underlying languages, i.e. an algorithm can be implemented in more than one programming language.

From the data structure point of view, following are some important categories of algorithms −

- **Search** − Algorithm to search an item in a data structure.
- **Sort** − Algorithm to sort items in a certain order.
- **Insert** − Algorithm to insert item in a data structure.
- **Update** − Algorithm to update an existing item in a data structure.
- **Delete** − Algorithm to delete an existing item from a data structure.

Characteristics of an Algorithm

Not all procedures can be called an algorithm. An algorithm should have the following characteristics −

- **Unambiguous** − Algorithm should be clear and unambiguous. Each of its steps (or phases), and their inputs/outputs should be clear and must lead to only one meaning.
- **Input** − An algorithm should have 0 or more well-defined inputs.
- **Output** − An algorithm should have 1 or more well-defined outputs, and should match the desired output.
- **Finiteness** − Algorithms must terminate after a finite number of steps.
- **Feasibility** − Should be feasible with the available resources.
- **Independent** − An algorithm should have step-by-step directions, which should be independent of any programming code.

How to Write an Algorithm?

There are no well-defined standards for writing algorithms. Rather, it is problem and resource dependent. Algorithms are never written to support a particular programming code.

As we know that all programming languages share basic code constructs like loops (do, for, while), flow-control (if-else), etc. These common constructs can be used to write an algorithm.

We write algorithms in a step-by-step manner, but it is not always the case. Algorithm writing is a process and is executed after the problem domain is well-defined. That is, we should know the problem domain, for which we are designing a solution.

Example : **Let's try to learn algorithm-writing by using an example.**

Problem − Design an algorithm to add two numbers and display the result.

Step 1 − START

Step 2 − declare three integers **a**, **b** & **c**

Step 3 − define values of **a** & **b**

Step 4 − add values of **a** & **b**

Step 5 − store output of step 4 to **c**

Step 6 − print **c**

Step 7 − STOP

Algorithms tell the programmers how to code the program. Alternatively, the algorithm can be written as −

Step 1 − START ADD

Step 2 − get values of **a** & **b**

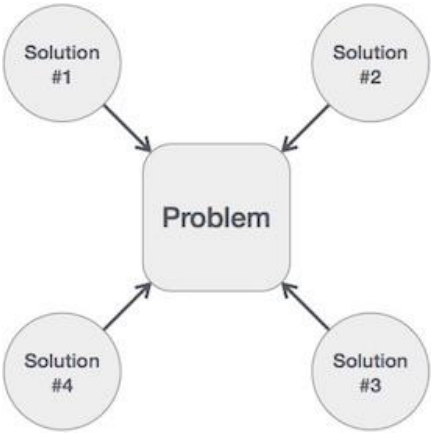
Step 3 − c ← a + b

Step 4 − display c

Step 5 − STOP

In design and analysis of algorithms, usually the second method is used to describe an algorithm. It makes it easy for the analyst to analyze the algorithm ignoring all unwanted definitions. He can observe what operations are being used and how the process is flowing. Writing **step numbers**, is optional.

We design an algorithm to get a solution of a given problem. A problem can be solved in more than one ways.



Hence, many solution algorithms can be derived for a given problem. The next step is to analyze those proposed solution algorithms and implement the best suitable solution.

Algorithm Analysis

Efficiency of an algorithm can be analyzed at two different stages, before implementation and after implementation. They are the following −

- ***A Priori* Analysis** − This is a theoretical analysis of an algorithm. Efficiency of an algorithm is measured by assuming that all other factors, for example, processor speed, are constant and have no effect on the implementation.
- ***A Posterior* Analysis** − This is an empirical analysis of an algorithm. The selected algorithm is implemented using programming language. This is then executed on target computer machine. In this analysis, actual statistics like running time and space required are collected.

We shall learn about *a priori* algorithm analysis. Algorithm analysis deals with the execution or running time of various operations involved. The running time of an operation can be defined as the number of computer instructions executed per operation.

Algorithm Complexity

Suppose **X** is an algorithm and **n** is the size of input data, the time and space used by the algorithm X are the two main factors, which decide the efficiency of X.

- **Time Factor** − Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.
- **Space Factor** − Space is measured by counting the maximum memory space required by the algorithm.

The complexity of an algorithm **f(n)** gives the running time and/or the storage space required by the algorithm in terms of **n** as the size of input data.

Space Complexity

Space complexity of an algorithm represents the amount of memory space required by the algorithm in its life cycle. The space required by an algorithm is equal to the sum of the following two components −

- A fixed part that is a space required to store certain data and variables, that are independent of the size of the problem. For example, simple variables and constants used, program size, etc.
- A variable part is a space required by variables, whose size depends on the size of the problem. For example, dynamic memory allocation, recursion stack space, etc.

Space complexity S(P) of any algorithm P is S(P) = C + SP(I), where C is the fixed part and S(I) is the variable part of the algorithm, which depends on instance characteristic I. Following is a simple example that tries to explain the concept −

Algorithm: SUM(A, B)

Step 1 – START

Step 2 – C ← A + B + 10

Step 3 – Stop

Here we have three variables A, B, and C and one constant. Hence S(P) = 1 + 3. Now, space depends on data types of given variables and constant types and it will be multiplied accordingly.

Time Complexity

Time complexity of an algorithm represents the amount of time required by the algorithm to run to completion. Time requirements can be defined as a numerical function T(n), where T(n) can be measured as the number of steps, provided each step consumes constant time.

For example, addition of two n-bit integers takes **n** steps. Consequently, the total computational time is T(n) = c ∗ n, where c is the time taken for the addition of two bits. Here, we observe that T(n) grows linearly as the input size increases.